

- 机械臂自动化接触式绘图系统 (Automated Contact Drawing System)
 - Slide 1: 项目概览与核心挑战 (Overview & Challenges)
 - Slide 2: 轨迹工程与离线生成 (Trajectory Engineering)
 - Slide 3: 核心力觉感知与寻面 (Force Sensing & Auto-Touchdown)
 - Slide 4: 顶面绘制与安全控制 (Top-Down Drawing)
 - Slide 5: 侧边绘制 - 姿态变换与逆向重映射 (Side-Wall Drawing)
 - Slide 6: 侧边绘制 - 单轴法向力控与动摩擦对抗 (Force Control & Friction)
 - Slide 7: 轨迹对比工程与误差可视化 (Error Analysis & Visualization)
 - Slide 8: 结论与未来展望 (Conclusion & Future Work)

机械臂自动化接触式绘图系统 (Automated Contact Drawing System)

—— 顶面与侧边绘制、力觉感知及轨迹误差分析

Slide 1: 项目概览与核心挑战 (Overview & Challenges)

- **项目目标:** 实现机械臂 (Kortex Gen3 Lite) 对未知平面的自动化接触式轨迹绘制。
- **核心挑战:**
 1. 无末端六维力传感器, 如何实现精准的表面接触检测?
 2. 如何在未知平面 (水平面 / 垂直侧面) 上建立准确的轨迹映射?
 3. 接触作业中的动摩擦力、卡顿如何解决? 轨迹的执行精度如何评估?
- **解决方案:** 基于雅可比矩阵的力矩-力学转换、安全泄压控制策略、理论与实际轨迹的 3D/2D 对比误差分析系统。

Slide 2: 轨迹工程与离线生成 (Trajectory Engineering)

- **主要文件:** `trajectory/cube_circle_trajectory.py` (及各类形状脚本)
- **核心思想:** 独立于物理执行的 UV 坐标系轨迹生成, 将轨迹拆分为 `hover`、`touch_down`、`draw` 等相态。
- **核心代码片段:**

```
# 状态机分层：剥离物理执行，仅生成二维逻辑航点
for angle in np.linspace(0, 2 * math.pi, steps):
    x_m = center_x + radius_m * math.cos(angle)
    y_m = center_y + radius_m * math.sin(angle)
    # 为每一个点赋予相态，便于执行器动态调整高度
    waypoints.append({"x_m": x_m, "y_m": y_m, "phase": "draw"})
```

Slide 3: 核心力觉感知与寻面 (Force Sensing & Auto-Touchdown)

- 主要文件: `jacobian.py` (正运动学与雅可比计算)
- 核心思想: 利用公式 $F = (J^T)^+ \tau$ 将关节力矩转换为末端力估计，并进行动态零点去偏置。
- 核心代码片段:

```
# 在 joint_states_callback 中实时运算
thetas = msg.position[0:6]
torques = msg.effort[0:6]

# 计算基础雅可比矩阵，并通过伪逆将关节力矩转换为末端力
J = self.arm_model.basic_jacobian(thetas)
tool_force = np.linalg.pinv(J.T).dot(torques)

# 提取法向力并去除预先校准的静力偏置
raw_fz = tool_force[2]
self.current_fz = abs(raw_fz - self.fz_bias)
```

Slide 4: 顶面绘制与安全控制 (Top-Down Drawing)

- 主要文件: `tools/auto_contact_draw.py`
- 核心思想: 将 2D 轨迹映射到水平 XY 面，通过实时监测 Z 轴法向阻力进行“安全泄压”，避免机械臂卡死。
- 核心代码片段:

```
# 动态法向安全泄压策略 (Relief Strategy)
if wp['phase'] in ['draw', 'touch_down']:
    if fz_filtered > 15.0 or stuck_cnt > 10:
```

```

        z_offset_relief += 0.005 * dt # 阻力过大, 向 +Z 方向退缩
    elif fz_filtered < 8.0:
        z_offset_relief -= 0.002 * dt # 阻力变小, 恢复下压

# 维持固定接触深度
depth_offset = fixed_depth - z_offset_relief
target_z = contact_z - depth_offset

```

Slide 5: 侧边绘制 - 姿态变换与逆向重映射 (Side-Wall Drawing)

- 主要文件: `tools/align_wrist_side.py`, `tools/side_contact_draw.py`
- 核心思想: 控制末端姿态法向对齐 -Y 轴; 并将原平面 XY 轨迹重映射至立面 XZ。
- 核心代码片段:

```

# 坐标系倒转与重映射 (-dv)
# Z 轴控制上下: 取反原轨迹的 y 位移 (-dv), 实现“扎下后永远从下往上画”
# 这避免了笔尖由于重力和自锁效应导致的向下卡死
aligned_waypoints.append({
    'x': contact_x - du,
    'y': contact_y,
    'z': contact_z - dv,
    'phase': wp['phase']
})

```

Slide 6: 侧边绘制 - 单轴法向力控与动摩擦对抗 (Force Control & Friction)

- 主要文件: `tools/side_contact_draw.py`
- 核心思想: 侧面受到重力与垂直面的综合动摩擦, 通过实时读取目标面法向 (Y轴) 的受力来调节下压深度。
- 核心代码片段:

```

# 提取侧边绘制的法向受力 (Y轴)
raw_fy = tool_force[1]
self.current_fy = abs(raw_fy - self.fy_bias)

# 沿法向 (-Y) 的安全泄压退缩机制
if fy_filtered > 10.0 or stuck_cnt > 8:
    y_offset_relief += 0.005 * dt # 阻力过大, 向 +Y 泄压

```

```
elif fy_filtered < 5.0:  
    y_offset_relief -= 0.002 * dt # 恢复定深
```

Slide 7: 轨迹对比工程与误差可视化 (Error Analysis & Visualization)

- **主要文件:** `tools/side_contact_draw_with_log.py`, `tools/analyze_error.py`
- **核心思想:** 高频记录闭环实际轨迹，利用统计学自适应识别降维平面，展示物理执行与理论规划的闭合误差。
- **核心代码片段:**

```
# 通过理论目标点的方差，自动推断所在平面  
std_x, std_y, std_z = np.std(theo_x), np.std(theo_y), np.std(theo_z)  
stds = [('X', std_x, theo_x, act_x), ('Y', std_y, theo_y, act_y), ('Z', std_z, theo_z, act_z)]  
stds.sort(key=lambda item: item[1])  
plane_axes = stds[1:] # 剔除方差最小的法向，取出平面的两个主轴  
  
# 计算平移轨迹均方根误差 (RMSE)  
error = np.sqrt(np.mean((act_x - theo_x)**2 + (act_y - theo_y)**2 + (act_z - theo_z)**2))
```

Slide 8: 结论与未来展望 (Conclusion & Future Work)

- 成功在极有限的硬件条件（无末端独立力传感器）下，利用纯运动学和本体关节电流，实现了鲁棒性极强的自动化寻面和绘制。
- 建立了一套涵盖“轨迹生成 -> 力矩转换 -> 自适应安全控制 -> 数据记录 -> 自动误差可视化”的完整工程闭环。
- **误差分析结论:** 可视化中出现的闭环开口问题，本质是由于基础比例控制器 (P-Control) 在对抗相反方向的动摩擦力时产生的**方向性稳态滞后误差**。
- **Future Work:** 计划在未来工作中探讨引入积分控制 (Integral Control) 以消除稳态误差彻底闭合图案，并进一步研究空间 3D 合力感知以应对复杂曲面接触。